LAPORAN PRAKTIKUM

TEKNOLOGI IoT II

KOMUNIKASI ASINKRON DAN WEBSOCKETS & TOPOLOGI JARINGAN TERDESENTRALISASI (ESP-NOW DAN MESH)

Disusun Oleh:

Salma Anggun Velyza (25782082)

TRI 3C

Dosen Pengajar:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

MODUL 4: INDIVIDU

Tujuan Praktikum:

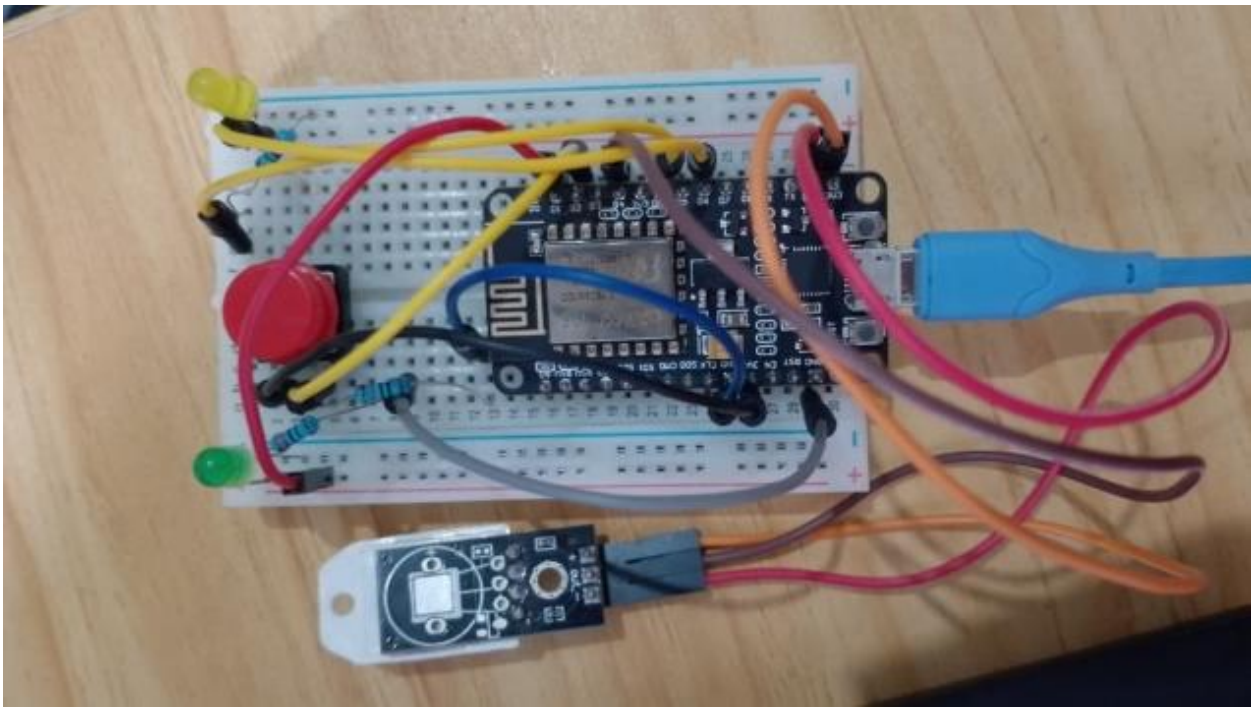
1. Memahami keterbatasan HTTP Web Server sinkron (masalah *blocking* dan *page refresh*).
2. Menggunakan pustaka ESPAsyncWebServer dan protokol *Full-Duplex* menggunakan *WebSockets*.
3. Membuat arsitektur sinkronisasi dua arah antara input perangkat keras (tombol fisik) dan antarmuka web *real-time*.
4. Mengimplementasikan kendali *PWM Slider* (Dimmer LED) via *WebSockets*.

Dasar Teori

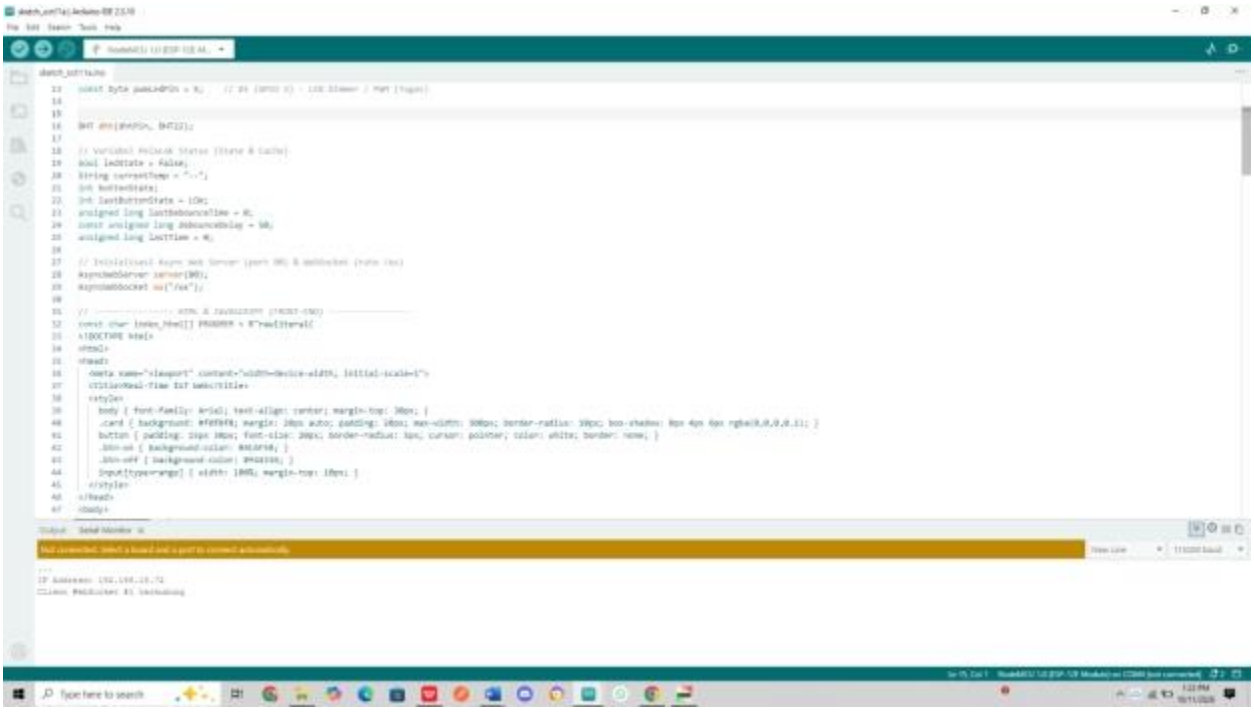
HTTP tradisional bersifat sinkron (*request-response*), di mana pembaruan data sensor memerlukan pemuatan ulang halaman web secara manual yang boros *bandwidth* dan menyebabkan transisi layar berkedip. *WebSockets* mengatasi hal ini dengan membuka koneksi pipa data persisten dua arah (*full-duplex*) antara *Client* (browser) dan *Server* (NodeMCU). Dengan arsitektur *push data*, NodeMCU dapat langsung mendorong paket data JSON ke *browser* seketika saat terjadi perubahan nilai sensor atau penekanan tombol fisik tanpa perlu *refresh* halaman.

Alat dan Bahan

1. Board NodeMCU ESP8266 (1 buah)
2. Kabel Micro USB (1 buah)
3. Breadboard (1 buah)
4. LED 5mm (2 buah: LED Utama & LED Dimmer)
5. Resistor 220 Ohm (2 buah)
6. Push Button / Tactile Switch (1 buah)
7. Resistor 10k Ohm (1 buah untuk *pull-down* tombol)
8. Sensor Suhu & Kelembapan DHT22 (1 buah)
9. Kabel Jumper secukupnya



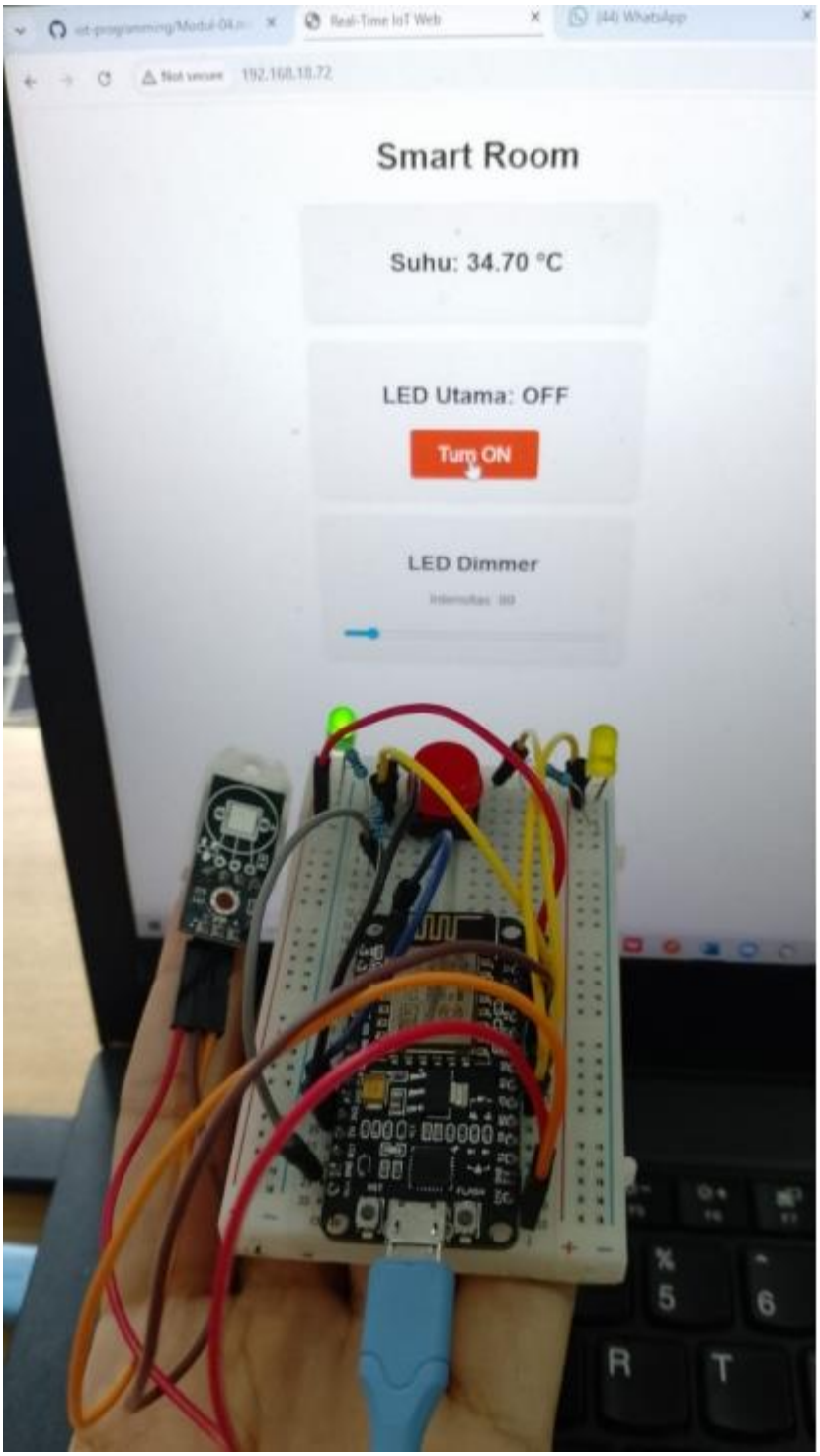
Gambar Rangkaian Perangkat Keras (NodeMCU, Sensor DHT22, Push Button, dan dua LED).



Tangkapan Layar Serial Monitor Arduino IDE.

Menampilkan alamat IP lokal perangkat (`192.168.18.72`) serta status koneksi WebSocket klien yang berhasil terhubung.

Foto Kondisi MATI (OFF):



Menampilkan layar web saat tombol menunjukkan status `LED Utama: OFF` (warna tombol merah) dan lampu LED di *breadboard* dalam keadaan padam.

Foto Kondisi HIDUP (ON):



Menampilkan layar web saat tombol menunjukkan status LED Utama: ON (warna tombol hijau), tuas *slider* PWM aktif, dan lampu LED di *breadboard* menyala terang.

Hasil Percobaan dan Pembahasan

1. **Serial Monitor:** Saat program dijalankan, Serial Monitor menampilkan alamat IP lokal (`IP Address: 192.168.18.72`) dan status `Client WebSocket terhubung` yang menandakan koneksi jaringan berjalan sukses.
2. **Dashboard Web:** Akses melalui peramban web menampilkan antarmuka *Smart Room* yang memuat informasi suhu dari sensor DHT22, tombol interaktif LED utama, serta tuas *slider* untuk mengontrol intensitas cahaya LED Dimmer.
3. **Pengujian Sinkronisasi:** Saat tombol fisik ditekan atau tombol di antarmuka web diklik, status indikator menyala/mati secara sinkron dan instan (*real-time*) tanpa jeda *loading*. Pergeseran tuas *slider* pada layar web juga berhasil meredupkan atau menerangkan LED di *breadboard* secara bertahap.

Kesimpulan Penggunaan protokol *WebSockets* dan *Async Web Server* pada ESP8266 sangat efektif dalam membangun sistem IoT modern. Pemrograman *non-blocking* menggunakan fungsi `millis()` memungkinkan mikrokontroler memproses interaksi jaringan dan pembacaan sensor secara bersamaan tanpa mengalami *delay* atau *freeze*, sehingga menghasilkan respons perangkat yang cepat dan efisien.

MODUL 5: KELOMPOK

Tujuan Praktikum

- Membangun jaringan *Ad-Hoc Mesh* otonom menggunakan painlessMesh tanpa ketergantungan *router*.
- Mengimplementasikan pengiriman data sensor lingkungan (DHT dan LDR) dalam format JSON lintas node.
- Menguji sistem *Rule Engine* pada Node Konsentrator (Hub) untuk otomasi aktuator LED.

Alat dan Bahan

- 2 Unit Board NodeMCU ESP8266.
- Sensor Suhu & Kelembapan (DHT11 / DHT22).
- Sensor Cahaya (LDR) dan resistor 10k Ω.
- LED 5mm dan Resistor 220 Ω.
- Kabel *jumper* dan *breadboard*.

Langkah Kerja & Pembagian Tugas Kelompok

- **Salma (Node 2):** Bertindak sebagai pengirim data sensor cahaya (*LDR*) menggunakan pin analog **A0**.
- **Riri (Node 1 + Hub):** Bertindak sebagai pengirim data suhu (*DHT*) di pin **D4** sekaligus pusat kendali (*Rule Engine*) yang mengontrol aktuator LED di pin **D6**.

Kode node 1 dan 3

```
void setup() {
  // put your setup code here, to run once:

}

void loop() {
  // put your main code here, to run repeatedly:

}

#include <painlessMesh.h>
#include <DHT.h>
#include <ArduinoJson.h>

#define MESH_PREFIX   "LabIoTMesh"
#define MESH_PASSWORD "iotmeshpassword"
#define MESH_PORT     5555

// DHT22
#define DHTPIN  2       // OUT ke D4 (GPIO 2)
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

// LED Node 3
#define LED_PIN 12      // D6 (GPIO 12)

const char *nodeName = "Node-1+3";

Scheduler userScheduler;
painlessMesh mesh;

void sendMessage();

Task taskSendMessage(
  TASK_SECOND * 3,
  TASK_FOREVER,
  &sendMessage
);

void sendMessage() {
  float suhu = dht.readTemperature();
```

```
  float kelembapan = dht.readHumidity();

  if (isnan(suhu) || isnan(kelembapan)) {
    Serial.println("Gagal membaca DHT22! Cek kabel.");
    return;
  }

  // Rule engine LED berdasarkan suhu
  if (suhu > 31.0) {
    digitalWrite(LED_PIN, HIGH);
    Serial.println("LED MENYALA: suhu > 31 C");
  } else {
    digitalWrite(LED_PIN, LOW);
    Serial.println("LED MATI: suhu normal");
  }

  // Format JSON telemetri
  StaticJsonDocument<200> doc;
  doc["tipe"] = "suhu_node";
  doc["node"] = nodeName;
  doc["suhu"] = suhu;
  doc["kelembapan"] = kelembapan;

  String msg;
  serializeJson(doc, msg);

  // Siarkan data melalui mesh
  mesh.sendBroadcast(msg);

  Serial.println("Data terkirim: " + msg);
}

void receivedCallback(uint32_t from, String &msg) {
  Serial.printf("Pesan dari node %u: %s\n",
                from, msg.c_str());
}

void newConnectionCallback(uint32_t nodeId) {
  Serial.printf("Node terhubung: %u\n", nodeId);
}

void setup() {
  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);
```

```
  digitalWrite(LED_PIN, LOW);

  dht.begin();

  mesh.setDebugMsgTypes(ERROR | STARTUP);
  mesh.init(
    MESH_PREFIX,
    MESH_PASSWORD,
    &userScheduler,
    MESH_PORT
  );

  mesh.onReceive(&receivedCallback);
  mesh.onNewConnection(&newConnectionCallback);

  userScheduler.addTask(taskSendMessage);
  taskSendMessage.enable();

  Serial.println("Node gabungan DHT22 + LED siap");
}

void loop() {
  mesh.update();
}
```

Kode node 2:

```
#include <painlessMesh.h>
#include <ArduinoJson.h>

#define MESH_PREFIX     "LabIoTMesh"
#define MESH_PASSWORD   "iotmeshpassword"
#define MESH_PORT       5555

const char* nodeName = "Node-2-Cahaya";

Scheduler userScheduler;
painlessMesh mesh;

void sendMessage();
Task taskSendMessage(TASK_SECOND * 3, TASK_FOREVER, &sendMessage);

void sendMessage() {
  // Membaca nilai analog dari pin AO modul LDR
  int nilaiLDR = analogRead(A0);

  // Bungkus data ke format JSON sesuai modul
  StaticJsonDocument<200> doc;
  doc["tipe"] = "cahaya_node";
  doc["adc"] = nilaiLDR;

  String msg;
  serializeJson(doc, msg);

  // Kirim data ke jaringan mesh
  mesh.sendBroadcast(msg);

  Serial.print("[KIRIM MESH - LDR] ");
  Serial.println(msg);
}

void receivedCallback(uint32_t from, String &msg) {
  StaticJsonDocument<200> doc;
  DeserializationError error = deserializeJson(doc, msg);

  if (!error) {
    const char* tipe = doc["tipe"];
    Serial.printf("[TERIMA MESH] Dari Node: %u | Tipe: %s\n", from, tipe);
```

```
  } else {
    Serial.printf("[TERIMA MENTAH DARI %u]: %s\n", from, msg.c_str());
  }
}

void newConnectionCallback(uint32_t nodeId) {
  Serial.printf("--> Koneksi Baru Terdeteksi! Node ID: %u\n", nodeId);
}

void changedConnectionCallback() {
  Serial.println("--> Topologi rantai mesh telah diperbarui");
}

void setup() {
  Serial.begin(115200);

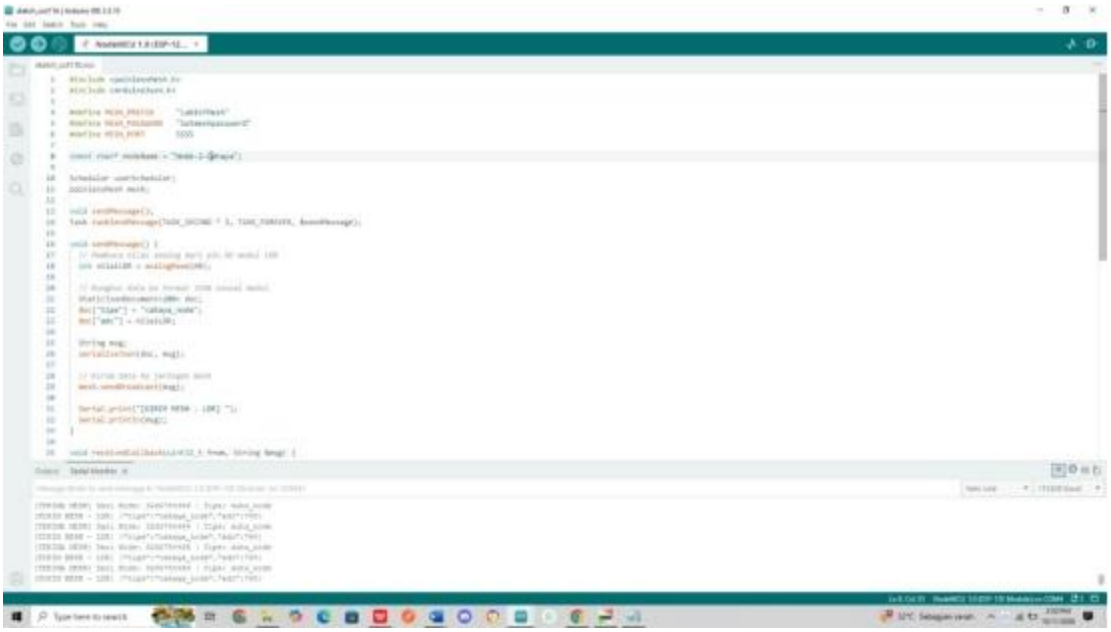
  mesh.setDebugMsgTypes(ERROR | STARTUP);
  mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);

  mesh.onReceive(&receivedCallback);
  mesh.onNewConnection(&newConnectionCallback);
  mesh.onChangedConnections(&changedConnectionCallback);

  userScheduler.addTask(taskSendMessage);
  taskSendMessage.enable();

  Serial.printf("Node LDR [%s] Berjalan...\n", nodeName);
}

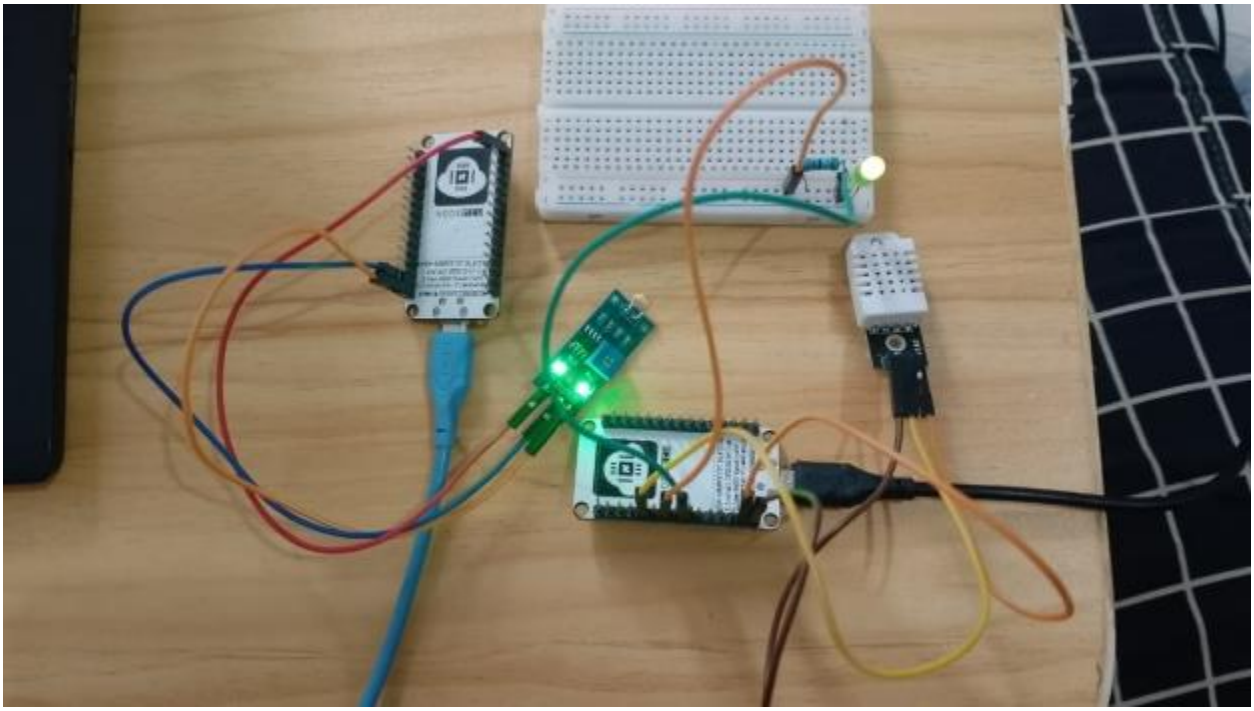
void loop() {
  mesh.update();
}
```



Tangkapan layar Serial Monitor pada Node 2 (Stasiun Cahaya LDR) yang menunjukkan proses pembacaan nilai sensor cahaya analog dan pengiriman paket data secara berkala ke jaringan *mesh* dalam format JSON ({"tipe":"cahaya_node","adc":765}).



Gambar X.2: Tangkapan layar Serial Monitor pada Node 1 & Hub Aktuator (Punya Riri) yang menampilkan proses penerimaan paket data telemetri dari node LDR milik rekan kelompok secara nirkabel via jaringan *mesh*, sekaligus eksekusi logika *Rule Engine* yang memicu status aktuator LED (LED NYALA: suhu > 31 C) secara otomatis ketika suhu lingkungan melampaui ambang batas (*threshold*).



angkaian fisik lengkap Node 1 (DHT), Node 2 (LDR), dan Node 3 Hub Aktuator LED pada breadboard.

Analisis dan Kesimpulan

- **Analisis:** Jelaskan bahwa jaringan *mesh* berhasil terbentuk secara otonom (*self-organizing*), data telemetri berhasil dikirimkan dalam bentuk format JSON, dan *Rule Engine* di sisi Hub sukses menyalakan LED secara otomatis saat kondisi suhu atau cahaya memenuhi ambang batas.
- **Kesimpulan:** Jaringan terdesentralisasi *painlessMesh* sangat efektif digunakan untuk sistem monitoring jarak jauh tanpa memerlukan infrastruktur *router* fisik di lapangan.